Lecture 11

Airflow + Ollama

Orchestrate extraction from a **weather API**, then use **Ollama** to produce **structured JSON**

Where this fits

Piece	Role
Airflow	**When** tasks run, retries, monitoring, dependencies
Weather API	**Source** — live, semi-structured JSON (Open-Meteo, no key)
Ollama	**Transform** — LLM turns raw text into a **fixed schema**
Validation task	**Contract** — fail the pipeline if JSON is incomplete

Lecture 10: run Ollama as a **service** (Nomad or local). **Lecture 11:** call it from **DAG tasks**.

Unstructured vs structured (for this lab)

- **Raw API body** — correct JSON, but **nested**, many keys, codes (e.g. WMO `weather_code`).
- **“Structured” for analytics” — one flat JSON** with stable field names your dashboards / warehouse expect.

The LLM’s job: **normalize** and **explain** (short `conditions_short`) while **filling a schema**.

Why use a LLM here?

- **Rapid prototyping** — change the schema in a prompt instead of hand-writing parsers for every vendor field.
- **Messy sources** — HTML, PDF snippets, inconsistent JSON; models can map to a target shape (with validation downstream).
- **Cost / privacy** — **Ollama** runs **on your machine** or your cluster (see Lecture 10 + Nomad).

Trade-off: models can **hallucinate** → always **validate** outputs (Airflow task or Great Expectations, etc.).

Airflow recap (minimal)

- **DAG** — workflow definition; **tasks** — units of work.
- **TaskFlow (`@task`)** — Python functions as tasks; **XCom** passes return values.
- **Operators** — `PythonOperator`, **HTTP** hooks, etc.
- **Variables / Connections** — config without hardcoding (`ollama_base_url`, `ollama_model`).

Ollama HTTP API (pattern)

- **`POST /api/chat`** — messages + model; optional **`"format": "json"`** for JSON-shaped replies (Ollama version dependent).
- **Base URL** — `http://127.0.0.1:11434` locally; **`http://host.docker.internal:11434`** if Airflow runs in Docker on Mac.

Assignment DAG: `fetch` → **`ollama_to_structured`** → **`validate_and_emit`**.

Open-Meteo (assignment source)

- **HTTPS** forecast API — **no API key**
- Example: latitude / longitude + `current` + `daily` parameters
- Returns JSON; first task stores it as a **string** for the LLM (treat as “raw payload”)

Docs: open-meteo.com

Pipeline shape (ETL mindset)

1. **Extract** — HTTP GET weather JSON.
2. **Transform** — LLM prompt: “Return **only** JSON with keys: …”.
3. **Load / validate** — `json.loads`, check required keys; extend later to **S3 / Postgres**.

Airflow owns **scheduling** and **lineage**; Ollama owns **interpretation**.

Reliability & ops

- **Retries** on HTTP (weather + Ollama).
- **Timeouts** — model pulls can be slow; set generous `timeout` on `requests.post`.
- **Mock env** — `WEATHER_PIPELINES_MOCK_OLLAMA=1` skips Ollama for CI (see assignment).
- **Secrets** — do not commit API keys; Open-Meteo demo needs none.

Lecture 10 → 11

- **L10:** Package **Ollama** (and optional UI) as **Nomad jobs**.
- **L11: Consume** Ollama from **Airflow** as part of a **data product** (weather → schema).

Same Ollama server — different **orchestrator** (Nomad for *service placement*, Airflow for *pipeline runs*).

Assignment (summary)

- DAG `weather_unstructured_to_structured`
- Tasks: **fetch Open-Meteo → Ollama structured JSON → validate keys**
- Submit: screenshots + DAG file

See `LECTURE11_ASSIGNMENT_README.md`.

Troubleshooting (short)

Symptom	Check
Connection refused to Ollama	`ollama serve`, URL, Docker `host.docker.internal`
Invalid JSON from model	Use **`llama3.2`** or stricter prompt; add validation task (already there)
Airflow cannot see DAG	`AIRFLOW_HOME/dags`, file name, import errors in scheduler logs

Summary

- **Airflow** orchestrates **weather fetch → LLM → validation**.
- **Ollama** implements **unstructured / messy → structured** with a **schema contract**.
- **Validate** LLM output in code — treat the model as an **untrusted transformer**.

Next: Run the assignment DAG locally (or mock), capture logs, open PR.